Python Lesson Notes: Dictionaries

Introduction

So far, you have learned about several types of data in Python such as:

- Numbers (integers and floats)
- Strings (text)
- Lists

These data types store values, but sometimes we want to organize information in a way where **one value is associated with another value**.
For example, we may want to store:

- A student's name and their grade
- A person's name and their phone number
- A programming language and the editor used for it

To organize information like this, Python provides a data structure called a **dictionary**.

A **dictionary stores data as pairs consisting of a key and a value.**

What is a Dictionary?

A **dictionary** is a collection of **key–value pairs**.
Each **key** is connected to a **value**.

Example idea:

```
Name → Language
Naveen → Python
Kiran → Java
Harsh → JavaScript
```

In Python this relationship can be stored inside a dictionary.

1. Creating a Dictionary

Dictionaries are created using **curly braces {}**.

Basic syntax:

```python
data = {key1: value1, key2: value2, key3: value3}
```

Example:

```python
data = {1: "Naveen", 2: "Kiran", 4: "Harsh"}
```

Here:

Key	Value
1	Naveen
2	Kiran
4	Harsh

Important points:

- Each item is separated by a comma
- Each key is connected to a value using `:`

Important Characteristics of Dictionaries

Dictionaries behave differently from lists.

1. Dictionaries use keys instead of index numbers

Lists use numeric positions:

```
list[0]
list[1]
```

Dictionaries use **keys**:

```
data[key]
```

2. Keys must be unique

Each key can only appear once in a dictionary.
Example (not allowed conceptually):

```
{1:"A", 1:"B"}
```

The key `1` cannot appear twice.

3. Keys must be immutable

Keys must be a type that **cannot change**.

Examples of valid keys:

- strings
- numbers
- tuples

Examples of invalid keys:

- lists
- dictionaries

4. Values can be any data type

Values can be:

- numbers
- strings
- lists
- dictionaries
- other objects

Accessing Values from a Dictionary

To access a value, use the key inside square brackets.

Example:

```python
data = {1: "Naveen", 2: "Kiran", 4: "Harsh"}

print(data[4])
```

Output:

```
Harsh
```

Python looks for key `4` and returns its value.

5.What Happens if the Key Does Not Exist?

If you try to access a key that does not exist, Python produces an error.

Example:

```python
print(data[3])
```

Result:

```
KeyError
```

This means the key is not found in the dictionary.

6.Using the get() Method

To avoid errors when accessing keys, Python provides the .get() method.

Example:

```python
data.get(3)
```

Output:

```
None
```

This means the key was not found.

You can also specify a **default value**.

Example:

```python
data.get(3, "Not found")
```

Output:

```
Not found
```

This is useful when you are not sure if the key exists.

7.Adding a New Entry to a Dictionary

New items can be added simply by assigning a value to a new key.

Example:

```python
data["Monica"] = "C-Sharp"
```

Now the dictionary contains another key–value pair.

8.Deleting an Entry

Example:

```python
del data[4]
```

This removes:

```
4 : "Harsh"
```

from the dictionary.

9.Creating a Dictionary from Two Lists

Sometimes you have two lists:

One list contains keys.
The other list contains values.

Python provides the **zip() function** to combine them.

Example:

```python
keys = ["Naveen", "Kiran", "Harsh"]
values = ["Python", "Java", "JavaScript"]

data = dict(zip(keys, values))
```

Resulting dictionary:

```
{
 "Naveen": "Python",
 "Kiran": "Java",
 "Harsh": "JavaScript"
}
```

Each key is paired with the corresponding value.

10.Dictionaries with Lists as Values

Dictionaries behave differently from lists.

1. Dictionaries use keys instead of index numbers

Lists use numeric positions:

```
list[0]
list[1]
```

Dictionaries use **keys**:

```
data[key]
```

Method	Description	Behavior if Key Missing
dict[key]	Access value using key	Produces KeyError
dict.get(key)	Safely access value	Returns None
dict.get(key, default)	Access value with fallback	Returns default value
del dict[key]	Removes key-value pair	Produces error if key missing
dict.clear()	Removes all items	Dictionary becomes empty

Key Ideas to Remember

- A **dictionary stores key-value pairs**
- Dictionaries are created using **curly braces {}**
- Keys must be **unique**
- Keys must be **immutable**
- Values can be **any data type**
- .get() is a safer way to access values than using []
- Dictionaries can contain **lists or other dictionaries**